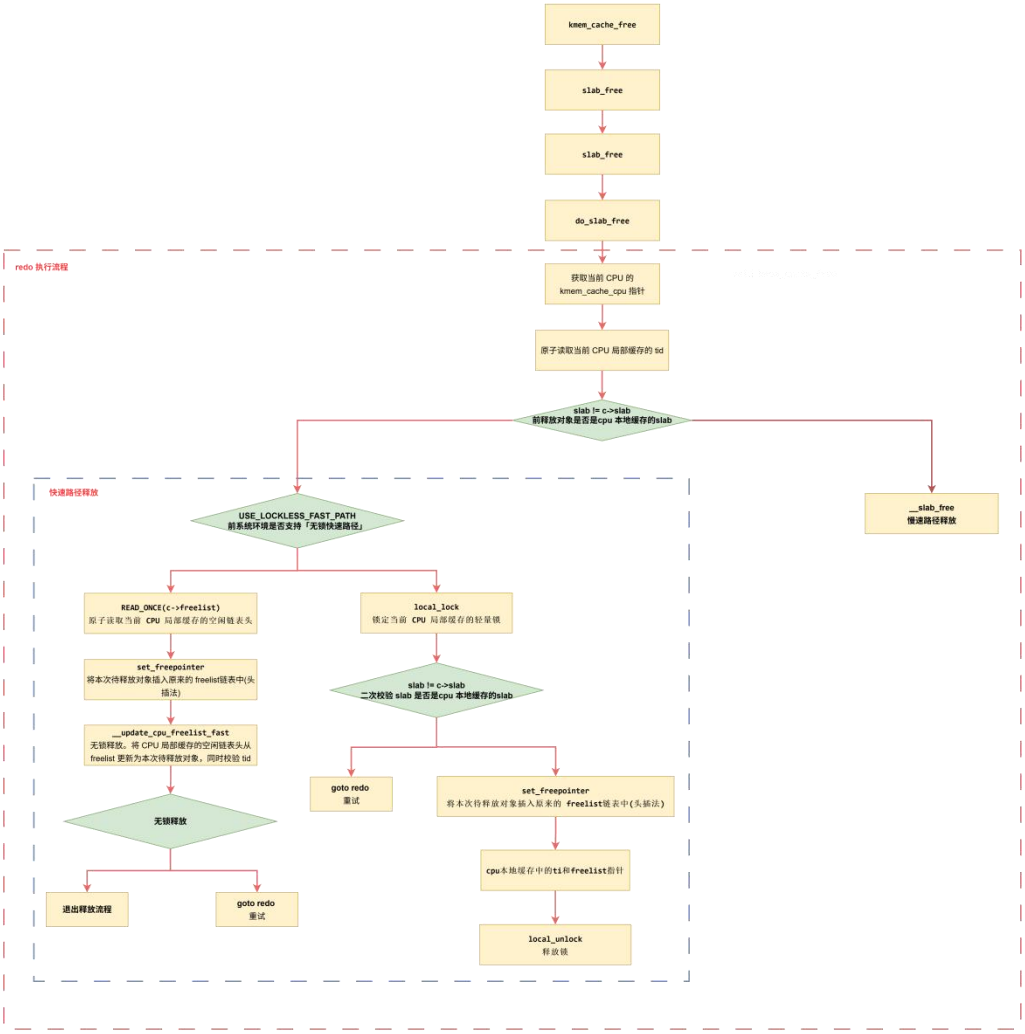
10 slab 内存池回收内存以及销毁流程 2
10.1 内存释放之前的校验工作 3
10.2 slab cache 快速路径回收内存 4
10.3 slab cache 慢速路径回收内存 8
10.2.1 释放对象回到 slab，调整 slab 相关属性 13
10.2.2 释放对象所属 slab 本来就在 cpu 缓存 partial 链表中 18
10.2.3 释放对象所属 slab 从 full slab 变为了 partial slab 18
10.3.4 释放对象所属 slab 从 partial slab 变为了 empty slab 23
10.3 销毁 slab cache 24
10.3.1 释放 slab cache 在各个 cpu 中的缓存资源 28
10.3.2 释放 slab cache 在各 node 节点中所有 partial 状态的 slab 30
10.3.2 释放 slab cache 的核心数据结构 32

10 slab 内存池回收内存以及销毁流程

内核中使用 kmem_cache_free 函数，将对象释放回其所属的 slab cache 中，参数 x 表示要释放的内存块（对象）的虚拟内存地址，参数 s 指向内存块所属的 slab cache。

```
    /**
 * kmem_cache_free - Deallocate an object
 * @s: The cache the allocation was from.
 * @x: The previously allocated object.
 *
 * Free an object which was previously allocated from this
 * cache.
 */
void kmem_cache_free(struct kmem_cache *s, void *x)
{
    // 确保指定的是 slab cache : s 为对象真正所属的 slab cache
    s = cache_from_obj(s, x);
    if (!s)
        return;
    trace_kmem_cache_free(_RET_IP_, x, s);
    // 将对象释放会 slab cache 中
    slab_free(s, virt_to_slab(x), x, _RET_IP_);
}
EXPORT_SYMBOL(kmem_cache_free);
```

Slab 内存池回收内存整体流程图：



10.1 内存释放之前的校验工作

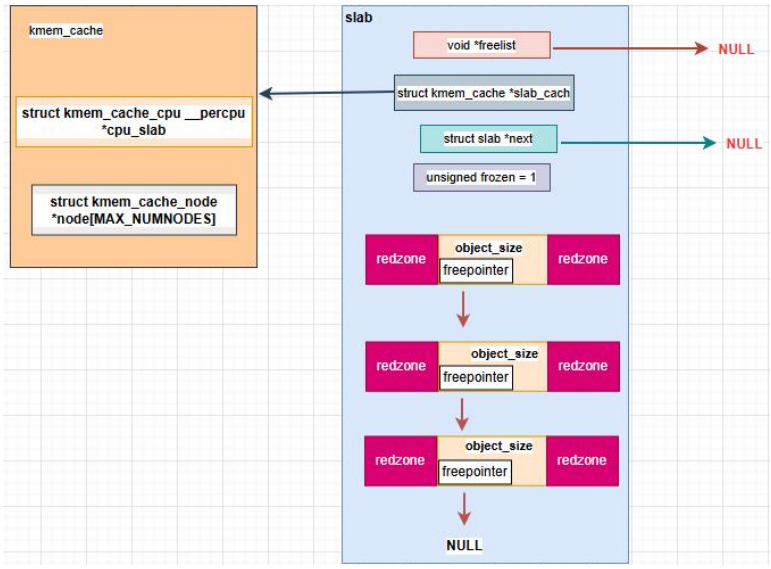
在开始释放内存块 x 之前，内核首先通过 cache_from_obj 函数确认内存块 x 是否真正属于指定的 slab cache。不能将内存块释放到其他的 slab cache 中。随后在 virt_to_head_page 函数中通过内存块的虚拟内存地址 x 找到其所在的物理内存页 page。然后调用 slab_free 将内存块释放回 slab cache 中。

```
static inline struct kmem_cache *cache_from_obj(struct kmem_cache *s, void
*x)
{
    struct kmem_cache *cachep;

    if (!IS_ENABLED(CONFIG_SLAB_FREELIST_HARDENED) &&
        !kmem_cache_debug_flags(s, SLAB_CONSISTENCY_CHECKS))
        return s;
    // 通过对象的虚拟内存地址 x 找到对象所属的 slab cache
    cachep = virt_to_cache(x);
```

```
    // 校验指定的 slab cache : s 是否是对象真正所属的 slab cache : cachep
    if (WARN(cachep && cachep != s,
         "%s: Wrong slab cache. %s but object is from %s\n",
         __func__, s->name, cachep->name))
        print_tracking(cachep, x);
    return cachep;
}
```

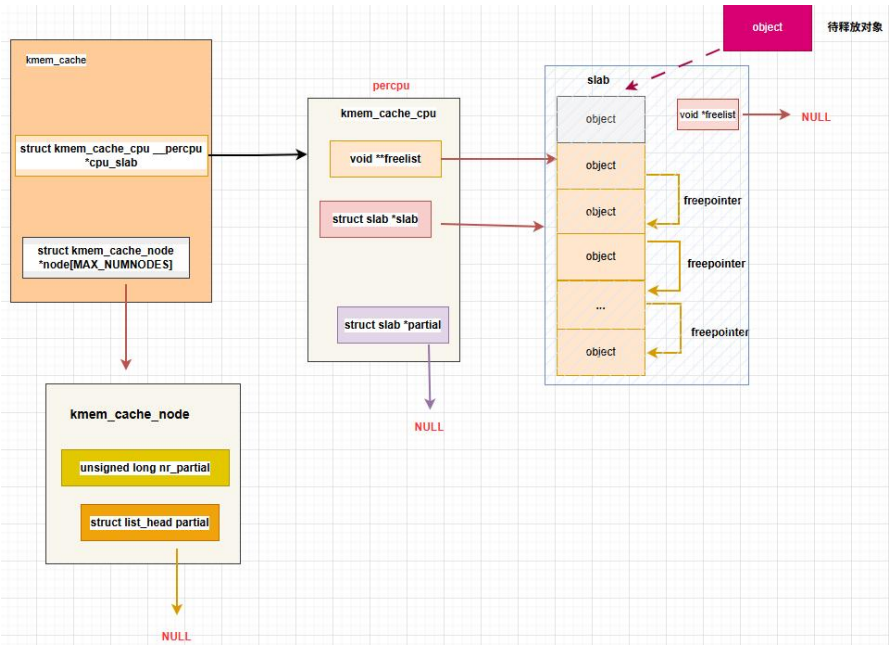
virt_to_cache 函数首先会通过释放对象的虚拟内存地址找到其所在的物理内存页 slab，然后通过 struct slab 结构中的 slab_cache 指针找到 slab 所属的 slab cache。




```
static inline struct kmem_cache *virt_to_cache(const void *obj)
{
    struct slab *slab;
     // 根据对象的虚拟内存地址 *obj 找到其所在的内存页 slab
     // 如果 slub 背后是多个内存页（复合页），则返回复合页的首页 head page
    slab = virt_to_slab(obj);
    if (WARN_ONCE(!slab, "%s: Object is not a Slab page!\n", __func__))
        return NULL;
     // 通过 slab 结构中的 slab_cache 属性找到其所属的 slub
    return slab->slab_cache;
}
```

10.2 slab cache 快速路径回收内存

内存回收总体也是分为快速路径 fastpath 和慢速路径 slow path，在 do_slab_free 函数中内核会首先尝试 fastpath 的回收流程。如果释放对象所在的 slab 刚好是 slab cache 在本地 cpu 缓存 kmem_cache_cpu->slab 缓存的 slab，那么内核就会直接将对象释放回缓存 slab 中。这时走的就是快速路径。




```
static __fastpath_inline
void slab_free(struct kmem_cache *s, struct slab *slab, void *object,
          unsigned long addr)
{
    memcg_slab_free_hook(s, slab, &object, 1);

    if (likely(slab_free_hook(s, object, slab_want_init_on_free(s))))
        do_slab_free(s, slab, object, object, 1, addr);
}
```

slab cache 回收内存相关的逻辑封装在 do_slab_free 函数中：

```
static __always_inline void do_slab_free(struct kmem_cache *s,
               struct slab *slab, void *head, void *tail,
               int cnt, unsigned long addr)
{
    struct kmem_cache_cpu *c;
     // slub 中对象分配与释放流程的全局事务 id
    //  既可以用来标识同一个分配或者释放的事务流程，
    // 也可以用来标识区分所属 cpu 本地缓存
    unsigned long tid;
    void **freelist;

redo:
    //获取 slab cache 的 cpu 本地缓存
    c = raw_cpu_ptr(s->cpu_slab);
     // 获取执行当前进程的 cpu 中的 tid 字段
    tid = READ_ONCE(c->tid);
    /* Same with comment on barrier() in slab_alloc_node() */
```

```
    barrier();
    if (unlikely(slab != c->slab)) {
        // 如果当前释放对象是 cpu 本地缓存 slab，
        // 那么就进入慢速释放路径 slowpath
        __slab_free(s, slab, head, tail, cnt, addr);
        return;
    }
    // 如果释放对象所属的 slub （page 表示）正好是 cpu 本地缓存的 slub
    // 那么直接将对象释放到 cpu 缓存的 slub 中即可，
    //这里就是快速释放路径 fastpath
    //USE_LOCKLESS_FAST_PATH: 标识当前系统环境是否支持「无锁快速路径」
    if (USE_LOCKLESS_FAST_PATH()) {
        freelist = READ_ONCE(c->freelist);
        // 将对象释放至 cpu 本地缓存 freelist 中的头结点处
       // 释放对象中的 freepointer 指向原来的 c->freelist
        set_freepointer(s, tail, freelist);
        if (unlikely(!__update_cpu_freelist_fast(s, freelist, head, tid)))
{
            note_cmpxchg_failure("slab_free", s, tid);
            goto redo;
        }
    } else {
        //轻量锁快速路径（无锁路径不支持时兜底）
        /* Update the free list under the local lock */
        local_lock(&s->cpu_slab->lock);
        c = this_cpu_ptr(s->cpu_slab);
        if (unlikely(slab != c->slab)) {
            local_unlock(&s->cpu_slab->lock);
            goto redo;
        }
        tid = c->tid;
        freelist = c->freelist;
        set_freepointer(s, tail, freelist);
        c->freelist = head;
        c->tid = next_tid(tid);
        local_unlock(&s->cpu_slab->lock);
    }
    stat_add(s, FREE_FASTPATH, cnt);
}
```

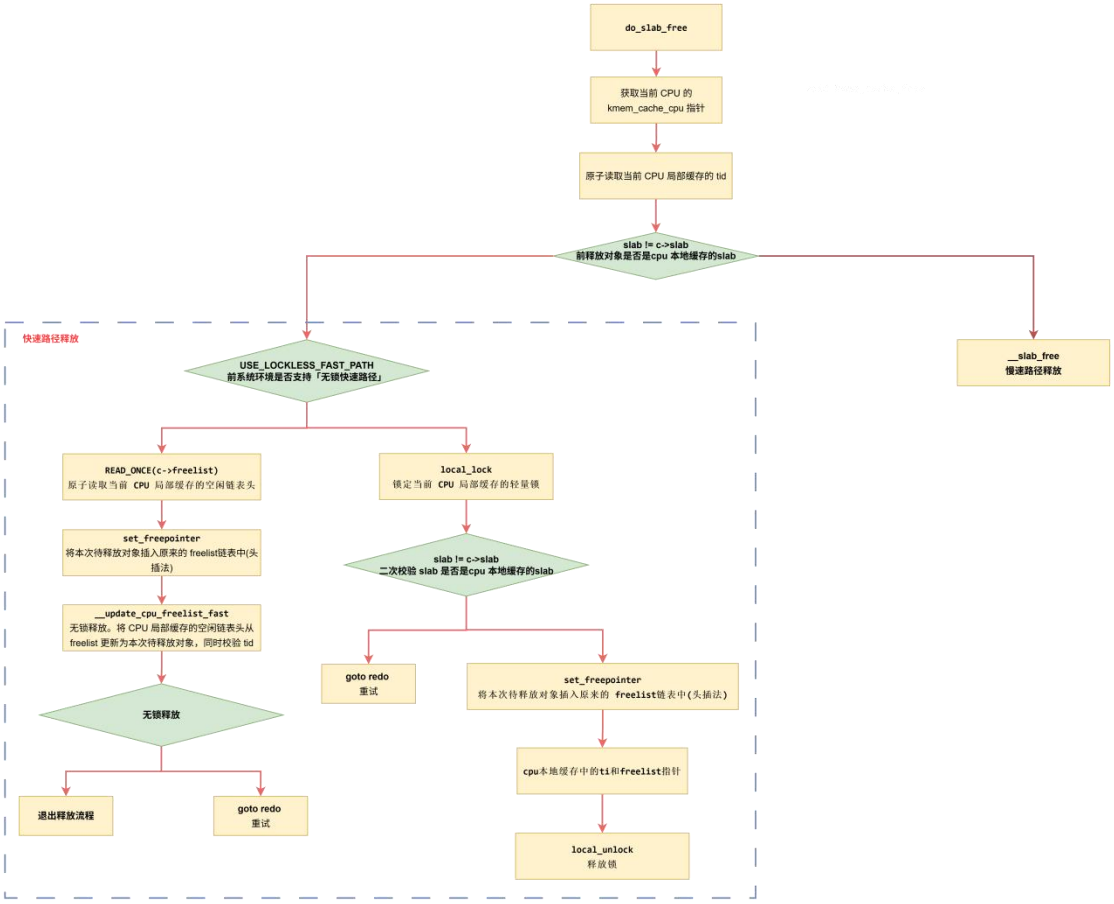
参数 kmem_cache *s 表示释放对象所在的 slab cache，指定要释放对象到哪里。

参数 slab 表示释放对象所在的内存页。

参数 head 指向释放对象的虚拟内存地址（起始内存地址）。该函数支持向 slab cache 批量的释放多个对象。

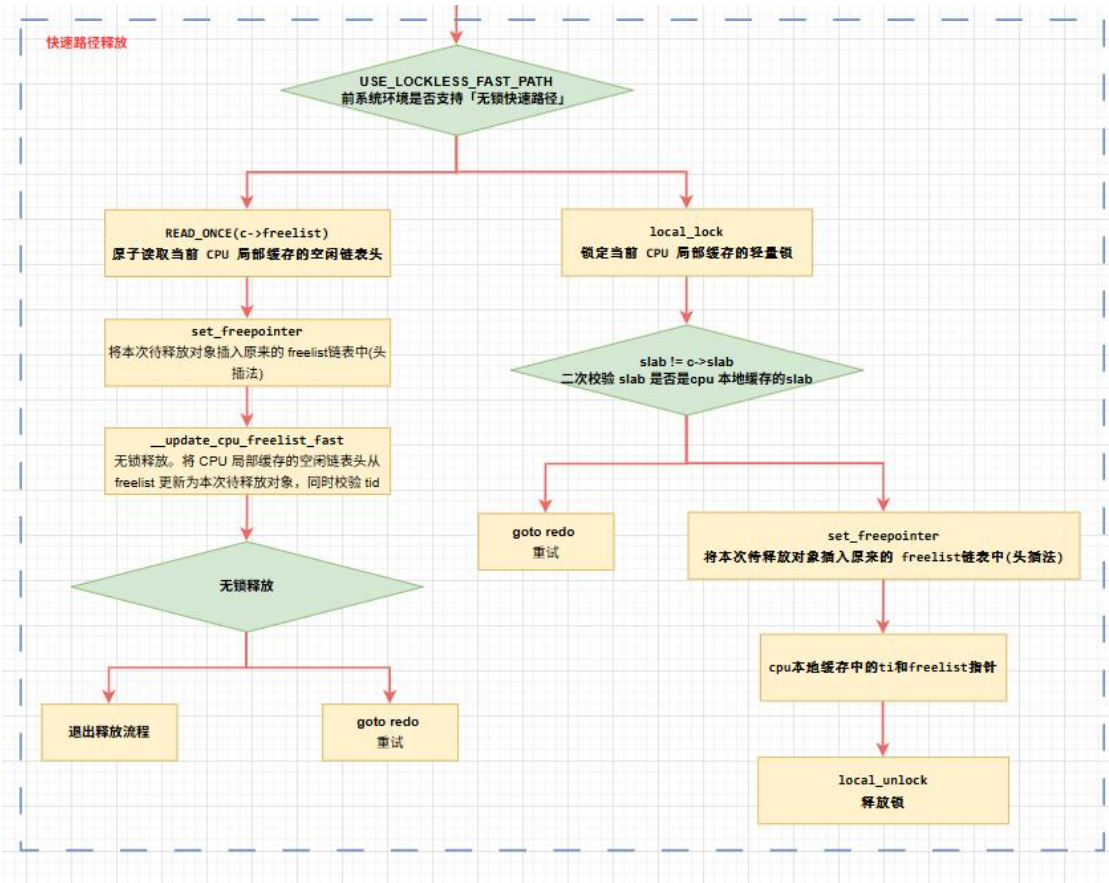
参数 tail 指向批量释放对象中最后一个对象的虚拟内存地址。

参数 cnt 表示释放对象的个数，也是用于批量释放对象。
参数 addr 用于 slab 调试。



内存回收总体也是分为快速路径 fastpath 和慢速路径 slow path，在 do_slab_free 函数的开始首先就获取 slab cache 的本地 cpu 缓存结构 kmem_cache_cpu。内核在获取 kmem_cache_cpu 结构之后，会判断该释放对象所在的 slab 是否正是 slab cache 本地 cpu 缓存了的 slab ——slab != c->slab。

如果释放对象所在的 slab 正是 slab cache 本地 cpu 缓存的 slab，进入快速释放路径，否则进入慢速释放路径。根据前 系统环境是否支持「无锁快速路径」，快速路径有分为无锁快速路径和有锁快速路径。



有锁快速路径，比无所快速路径锁两个逻辑：

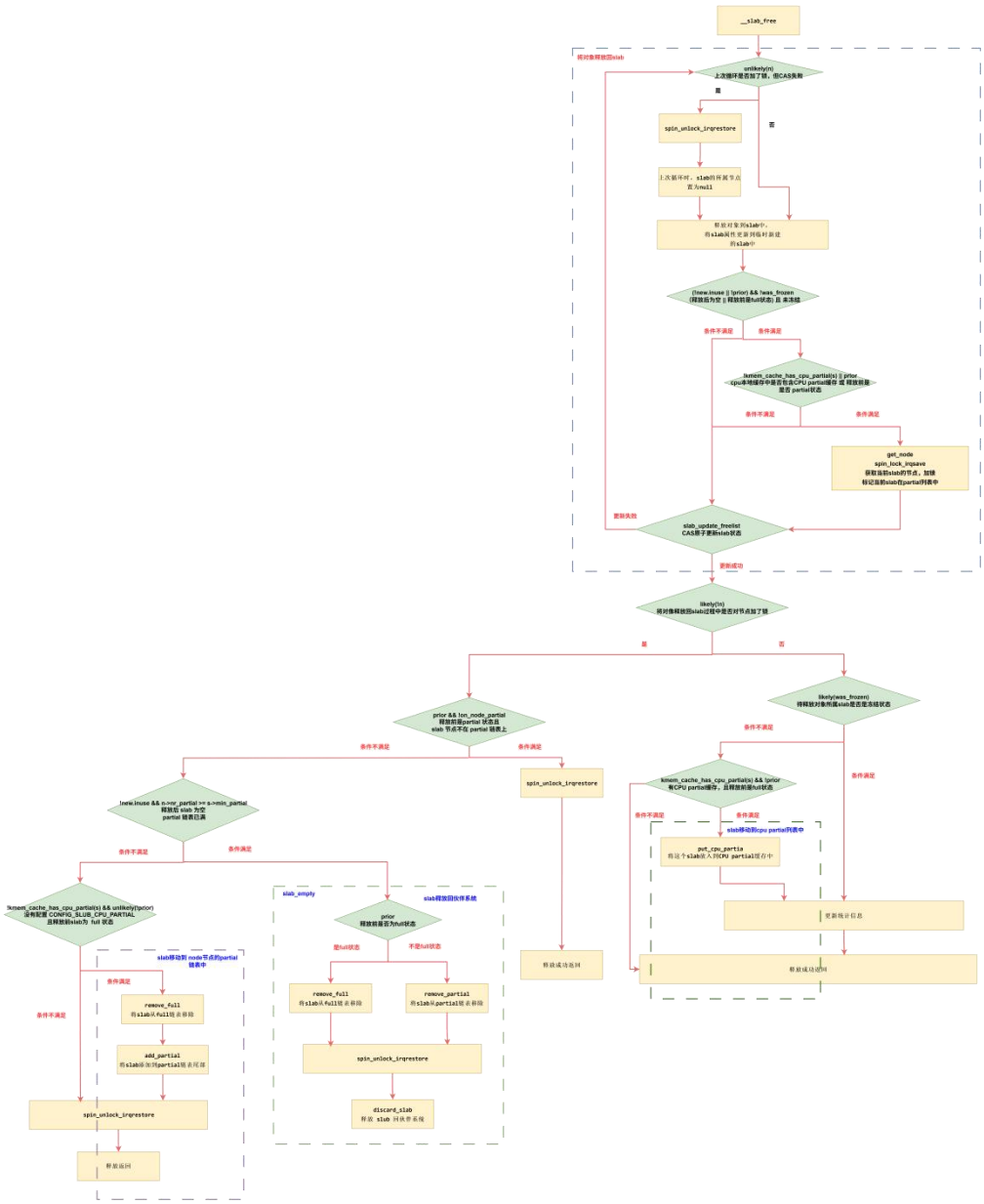
1、加锁逻辑

2、加锁后重新校验 slab 是否是 slab cache 本地 cpu 缓存的 slab

两者的都是直接将待释放的对象放回到 slab 中。

10.2 slab cache 慢速路径回收内存

slab cache 在慢速路径下回收内存需要改变释放对象所属 slab 在 slab cache 架构中的位置。在将对象释放回对应的 slab 中之前，内核需要首先清理一下对象所占的内存，重新填充对象的内存布局恢复到初始未使用状态。因为对象所占的内存此时包含了很多已经被使用过的无用信息。这项工作内核在 free_debug_processing 函数中完成。



在将对象所在内存恢复到初始状态之后，内核首先会将对象直接释放回其所属的 slab 中，并调整 slab 结构 slab 的相关属性。然后会改变 slab 在 slab cache 架构中的位置：

1. 如果 slab 本来就在 slab cache 本地 cpu 缓存 kmem_cache_cpu->partial 链表中，那么对象在释放之后，slab 的位置不做任何改变。
2. 如果 slab 不在 kmem_cache_cpu->partial 链表中，并且该 slab 由于对象的释放刚好由一个 full slab 变为了一个 partial slab，为了利用局部性的优势，内核需要将该 slab 插入到 kmem_cache_cpu->partial 链表中。**发生了 slab 节点的迁移**。
3. 如果 slab 不在 kmem_cache_cpu->partial 链表中，并且该 slab 由于对象的释放刚好由一个 partial slab 变为了一个 empty slab，说明该 slab 并不是很活跃，内核会将该 slab 放入对应 NUMA 节点缓存 kmem_cache_node->partial 链表中。**发生了 slab 节点的迁移。**
4. 如果不符合第 2, 3 种场景，但是 slab 本来就在对应的 NUMA 节点缓存 kmem_cache_node->partial 链表中，那么对象在释放之后，slab 的位置不做任何改变。

四种慢速释放场景封装在函数__slab_free 中。

```
static void __slab_free(struct kmem_cache *s, struct slab *slab,
            void *head, void *tail, int cnt,
            unsigned long addr)

{
   // 用于指向对象释放回 slub 之前，slub 的 freelist
    void *prior;
   // 对象所属的 slub 之前是否在本地 cpu 缓存 partial 链表中
   int was_frozen;
   // 后续会对 slub 对应的 slab 结构相关属性进行修改
   // 修改后的属性会临时保存在 new 中，后面通过 cas 替换
   struct slab new;
   unsigned long counters;
   struct kmem_cache_node *n = NULL;
   unsigned long flags;
   bool on_node_partial;
   stat(s, FREE_SLOWPATH);
   //两种场景直接调用 free_to_partial_list（加锁批量释放）。
   if (IS_ENABLED(CONFIG_SLUB_TINY) || kmem_cache_debug(s)) {
       free_to_partial_list(s, slab, head, tail, cnt, addr);
       return;
   }
   /*CAS 无锁重试循环（普通场景）,实现“无锁优先”的批量释放*
   do {
       //若上一次循环加了节点锁（n != NULL）但 CAS 失败，
       //先解锁并重置 n，避免重复加锁导致死锁。
       if (unlikely(n)) {
           spin_unlock_irqrestore(&n->list_lock, flags);
           n = NULL;
       }
       // 获取 slub 中的空闲对象列表，prior = null 表示此时
       // slub 是一个 full slub，意思就是该 slub 中的对象已经
       //全部被分配出去了
       prior = slab->freelist;
       counters = slab->counters;
       //将 tail（待释放对象）内部的 freepointer 字段，
       //赋值为 prior（slab 原有空闲链表的头指针），
       //从而实现「待释放对象」与「slab 原有空闲链表」的单向拼接。
       set_freepointer(s, tail, prior);
        // 将原有 slab 的相应属性赋值给 new slab
       new.counters = counters;
        // 获取原来 slub 中的 frozen 状态，
       //是否在 cpu 缓存 partial 链表中
```

```
      was_frozen = new.frozen;
      //计算释放后的已用对象数
       //inuse 表示 slub 已经分配出去的对象个数，
        //这里是释放 cnt 个对象，所以 inuse 要减去 cnt
      new.inuse -= cnt;
    //!new.inuse，  表示 slab 是一个 empty slab，里面的对象没有被使用
    //对象释放后 new.inuse 为 0，说明释放之前 slab 是 partial
    // !prior 表示此时 slab 是一个 full slab，已经没有空闲的对象了。
    //后续释放对象后会变成一个 partial slub
   // !was_frozen 表示该 slub 不在 cpu 本地缓存中
  /*三者的总体含义：（释放后为空 || 释放前是 full 状态) 且 未冻结*/
      if ((!new.inuse || !prior) && !was_frozen) {
           //如果配置了 CONFIG_SLUB_CPU_PARTIAL 选项，
           //那么表示 cpu 本地缓存 kmem_cache_cpu 结构中
           //包含 partial 列表，用于 cpu 缓存部分分配的 slub
          //进入该分支的 slub 之前都不在 cpu 本地缓存中
                if (!kmem_cache_has_cpu_partial(s) || prior) {
              //无 CPU partial 缓存 或 释放前是 partial 状态，进入此分支
             n = get_node(s, slab_nid(slab));
              //后续会操作 kmem_cache_node 中的 partial 列表，
             //所以这里需要获取 list_lock
               spin_lock_irqsave(&n->list_lock, flags);
             // 标记 slab 是否在节点 partial 链表
             on_node_partial = slab_test_node_partial(slab);
          }
      }
  } while (!slab_update_freelist(s, slab,
      prior, counters,
      head, new.counters,
      "__slab_free")); //CAS 原子更新 slab 状态，失败则重试循环
   //无锁分支：无需节点锁的释放完成
  /*
  1. (!new.inuse || !prior) && !was_frozen)加锁与判断条件不满足，
     可拆分为如下逻辑：
   1) slab 处于冻结状态（was_frozen = true），无需操作节点链表
     因为： 冻结的 slab（was_frozen = true）不参与 NUMA
     节点的 full/partial 链表迁移、空 slab 回收等操作，
     仅需完成对象的原子释放
   2) slab 处于未冻结状态（was_frozen = false）,但是，new.inuse != 0 且
       prior != NULL。其中，new.inuse != 0 表示释放后 slab 中 仍有
       已被使用的对象，此时处于 partial 状态。prior != NULL 表示
       释放前 slab 为 partial 状态，已有空闲对象。也就是说，这种情况下，
       slab 释放前后均为 partial 状态，且已挂载在节点 partial 链表上。
       无需迁移链表、无需回收 slab；
```

```
    */
    if (likely(!n)) {
        if (likely(was_frozen)) {
       // 分支 1：slab 是冻结状态，无需操作链表
            stat(s, FREE_FROZEN);
        } else if (kmem_cache_has_cpu_partial(s) && !prior) {
          //分支 2：有 CPU partial 缓存，且释放前是 full 状态，
          //放入 CPU partial 缓存
            put_cpu_partial(s, slab, 1);
            stat(s, CPU_PARTIAL_FREE);
        }
        return;
    }
    //加锁分支：无需修改节点链表（直接解锁返回）
    if (prior && !on_node_partial) {
      //prior != NULL（释放前是 partial 状态）且
      //!on_node_partial（slab 节点 不在 partial 链表上），
      // 无需修改节点链表（slab 不属于节点链表，仅更新了空闲链表），
      //直接解锁返回；
        spin_unlock_irqrestore(&n->list_lock, flags);
        return;
    }
     //加锁分支：判断是否需要回收空 slab
    //!new.inuse：释放后 slab 为空；
    //n->nr_partial >= s->min_partial：节点 partial 链表已满，
    //无需保留空 slab；
    if (unlikely(!new.inuse && n->nr_partial >= s->min_partial))
        goto slab_empty;
    //加锁分支：slab 从 full 转为 partial（链表迁移）
    if (!kmem_cache_has_cpu_partial(s) && unlikely(!prior)) {
        remove_full(s, n, slab);// 从 full 链表移除
          // 添加到 partial 链表尾部
        add_partial(n, slab, DEACTIVATE_TO_TAIL);
        stat(s, FREE_ADD_PARTIAL);
    }
    spin_unlock_irqrestore(&n->list_lock, flags);
    return;
slab_empty:
   // 该分支处理的场景是： slub 太多了，将 empty slub 释放会伙伴系统
  //首先将 slub 从对应的管理链表上删除
    if (prior) {
        remove_partial(n, slab);
        stat(s, FREE_REMOVE_PARTIAL);
    } else {
```

```
        remove_full(s, n, slab);
    }
    spin_unlock_irqrestore(&n->list_lock, flags);
    stat(s, FREE_SLAB);
     // 释放 slub 回伙伴系统，底层调用 __free_pages
    //将 slub 所管理的所有 slab 释放回伙伴系统
    discard_slab(s, slab);
}
```

10.3.1 将对象释放回 slab

将对象释放回其所属的 slab 中，并调整 slab 结构 slab 的相关属性的相关源码：

```
    /*CAS 无锁重试循环（普通场景）,实现 “无锁优先” 的批量释放*
    do {
        //若上一次循环加了节点锁（n != NULL）但 CAS 失败，
        //先解锁并重置 n，避免重复加锁导致死锁。
        if (unlikely(n)) {
            spin_unlock_irqrestore(&n->list_lock, flags);
            n = NULL;
        }
        // 获取 slub 中的空闲对象列表，prior = null 表示此时
        // slub 是一个 full slub，意思就是该 slub 中的对象已经
        //全部被分配出去了
        prior = slab->freelist;
        counters = slab->counters;
        //将 tail（待释放对象）内部的 freepointer 字段，
        //赋值为 prior（slab 原有空闲链表的头指针），
        //从而实现「待释放对象」与「slab 原有空闲链表」的单向拼接。
        set_freepointer(s, tail, prior);
         // 将原有 slab 的相应属性赋值给 new slab
        new.counters = counters;
         // 获取原来 slub 中的 frozen 状态，
        //是否在 cpu 缓存 partial 链表中
        was_frozen = new.frozen;
        //计算释放后的已用对象数
         //inuse 表示 slub 已经分配出去的对象个数，
          //这里是释放 cnt 个对象，所以 inuse 要减去 cnt
        new.inuse -= cnt;
      //!new.inuse，  表示 slab 是一个 empty slab，里面的对象没有被使用
      //对象释放后 new.inuse 为 0，说明释放之前 slab 是 partial
      // !prior 表示此时 slab 是一个 full slab，已经没有空闲的对象了。
      //后续释放对象后会变成一个 partial slub
```

```
    // !was_frozen 表示该 slub 不在 cpu 本地缓存中
   /*三者的总体含义：（释放后为空 || 释放前是 full 状态) 且 未冻结*/
      if ((!new.inuse || !prior) && !was_frozen) {
          //进入该分支的 slub 之前都不在 cpu 本地缓存中
                if (!kmem_cache_has_cpu_partial(s) || prior) {
              //无 CPU partial 缓存 或 释放前是 partial 状态，进入此分支
             n = get_node(s, slab_nid(slab));
              //后续会操作 kmem_cache_node 中的 partial 列表，
             //所以这里需要获取 list_lock
              spin_lock_irqsave(&n->list_lock, flags);
              //slab 是否已挂载在 NUMA 节点全局 partial 链表上
              //如果不是，则可能可能归属 CPU 局部 partial 缓存
              //、NUMA 节点 full 链表、已为空待回收或已销毁
               on_node_partial = slab_test_node_partial(slab);
          }
      }
  } while (!slab_update_freelist(s, slab,
      prior, counters,
      head, new.counters,
      "__slab_free")); //CAS 原子更新 slab 状态，失败则重试循环
```

在 __slab_free 内存释放流程的开始，首先会将对象直接释放回其所在的 slab 中。当对象被释放回 slab 中之后，slab 结构中的相应属于就需要做出相应的调整，比如：

调整 slab 结构中的 freelist，它需要指向刚刚被释放的对象。

调整 slab 结构中的 inuse，inuse 表示 slab 中已经被分配出去的对象个数，此时对象已经释放回 slab 中，需要调整 inuse 字段。

后续内核会根据不同情况，调整 slab 结构的 frozen 属性。

内核会定义一个新的 slab 结构 new，将原有 slab 的需要更新的上述属性的新值，先一一复制给 new 的对应属性，最后原子的更新 slab 对应的属性。

Slab 结构体中的 counter 属性和 inuse，frozen 属性被定义在一个 union 结构体中。union 结构体中定义的字段全部共享一片内存，union 结构体的内存占用由其中最大的属性决定。而 struct 结构体中的每个字段都是独占一片内存的。

由于 union 结构体中各个字段都是共享一块内存，所以一个字段的改变就会影响其他字段的值，从另一方面来看，通过一个字段就可以将整个 union 结构占用的内存块拿出来。

```
  struct slab {
        ......
                    union {
                    unsigned long counters;
                    struct {
                  // slab 中已经分配出去的对象
                        unsigned inuse:16;
                // slab 中包含的对象总数
                        unsigned objects:15;
                  // 该 slab 是否在对应 slab cache 的本地 CPU 缓存中
```

```
                    // frozen = 1 表示缓存再本地 cpu 缓存中
                            unsigned frozen:1;
                        };
                    };
            ......
};
```

slab 结构中的 counters 是和 inuse，frozen 共用同一块内存的，内核在 __slab_free 中将原有 slab 的 counters 属性赋值给 new.counters 的一瞬间，counters 所在的内存块也就赋值到 new slab 的 union 结构中了。counters 的地址就是这块内存块的地址。

而 inuse，frozen 属性的值也在这个内存块中，所以原有 slab 中的 inuse，frozen 属性也就跟着一起赋值到 new slab 的对应属性中了。

```
        counters = slab->counters;
         ......
        new.counters = counters;
        was_frozen = new.frozen;
        new.inuse -= cnt;
```

在原子的更新 slab 对应的属性操作中，根据缓存池配置（是否支持双字段原子比较交换），内核分别采取两种路不同径完成 slab->freelist（空闲链表头）和 slab->counters（状态计数器）的原子更新。分别是：快速无锁路径和慢速关中断路径。

```
static inline bool slab_update_freelist(struct kmem_cache *s, struct slab
*slab,
        void *freelist_old, unsigned long counters_old,
        void *freelist_new, unsigned long counters_new,
        const char *n)
{
    bool ret;
    //当前缓存池支持「双字段原子比较交换」
    //（即可以同时对 slab->freelist（指针类型）和
    // slab->counters（unsigned long 类型）两个独立字段，
    //执行 “先对比、后原子更新” 的操作）
    if (s->flags & __CMPXCHG_DOUBLE) {
    //快速路径：完成 slab->freelist（空闲链表头）
    //和 slab->counters（状态计数器）的原子更新。不许需要原子锁和中断操作。
        ret = __update_freelist_fast(slab, freelist_old, counters_old,
                            freelist_new, counters_new);
    } else {
        unsigned long flags;
        //慢速路径完成更新。
        //通过关闭本地中断，避免当前 CPU 被抢占，
        //从而在软件层面保证 “对比 + 更新” 的原子性
        local_irq_save(flags);
        ret = __update_freelist_slow(slab, freelist_old, counters_old,
```

```
                            freelist_new, counters_new);
        local_irq_restore(flags);
    }
    if (likely(ret))
      //更新生效，直接返回
        return true;
    cpu_relax();
    stat(s, CMPXCHG_DOUBLE_FAIL);
#ifdef SLUB_DEBUG_CMPXCHG
    pr_info("%s %s: cmpxchg double redo ", n, s->name);
#endif
    return false;
}
```

当前缓存池支持「双字段原子比较交换」时走快速路径：

```
static inline bool
__update_freelist_fast(struct slab *slab,
              void *freelist_old, unsigned long counters_old,
              void *freelist_new, unsigned long counters_new)
{
//system_has_freelist_aba 宏：的含义是「当前系统硬件架构支持 SLUB
//空闲链表的 ABA 问题防护」，同时支持「双字段连续内存的原子 CAS 指令」
#ifdef system_has_freelist_aba
    freelist_aba_t old = { .freelist = freelist_old, .counter =
counters_old };
    freelist_aba_t new = { .freelist = freelist_new, .counter =
counters_new };

    return try_cmpxchg_freelist(&slab->freelist_counter.full, &old.full,
new.full);
#else
    //直接返回 false，此时会退回到 slab_update_freelist 的慢速路径
    return false;
#endif
}
```

ABA 问题说明：若仅对 freelist 指针做 CAS 操作，可能出现“指针值从 A 变为 B，又变回 A”的场景，其他 CPU 会误以为 slab 状态未变更，导致 CAS 误判成功，引发内存混乱。引入 counter（递增计数器）后，即使 freelist 指针相同，counter 不同，CAS 也会失败，从而规避 ABA 问题。

否则，走慢速路径。通过关闭本地中断，避免当前 CPU 被抢占，实现原子的更新。

```
    else {
        unsigned long flags;
```

```
        //慢速路径完成更新。
        //通过关闭本地中断，避免当前 CPU 被抢占，
        //从而在软件层面保证“对比 + 更新”的原子性
        local_irq_save(flags);
        ret = __update_freelist_slow(slab, freelist_old, counters_old,
                                     freelist_new, counters_new);
        local_irq_restore(flags);
}
```

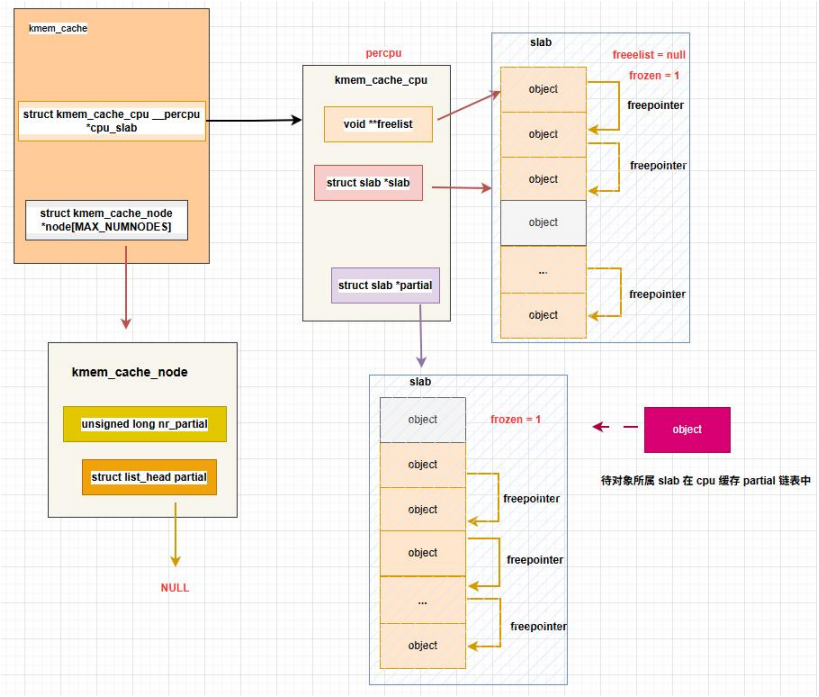
```
static inline bool
__update_freelist_slow(struct slab *slab,
             void *freelist_old, unsigned long counters_old,
             void *freelist_new, unsigned long counters_new)
{
    bool ret = false;

    slab_lock(slab);
    if (slab->freelist == freelist_old &&
        slab->counters == counters_old) {
        slab->freelist = freelist_new;
        slab->counters = counters_new;
        ret = true;
    }
    slab_unlock(slab);
    return ret;
}
```

不管快速路径还是慢速路径中，只是原子的更新了 slab 结构中的 freelist 和 counters ，并没有显示的更新 inuse，frozen 这两个属性。由于 counters，inuse，frozen 共用一块内存，当 inuse，frozen 的值发生变化之后，虽然 counters 的值没有发生变化，但是我们可以通过更新 counters 来将原有 slab 中的这块内存一起更新掉，这样 inuse，frozen 的值也跟着被更新了。

由于 slab 的 freelist 指针在 union 结构体之外，所以需要显示的单独更新。

10.3.2 释放对象所属的 slab 在 cpu 缓存 partial 链表中

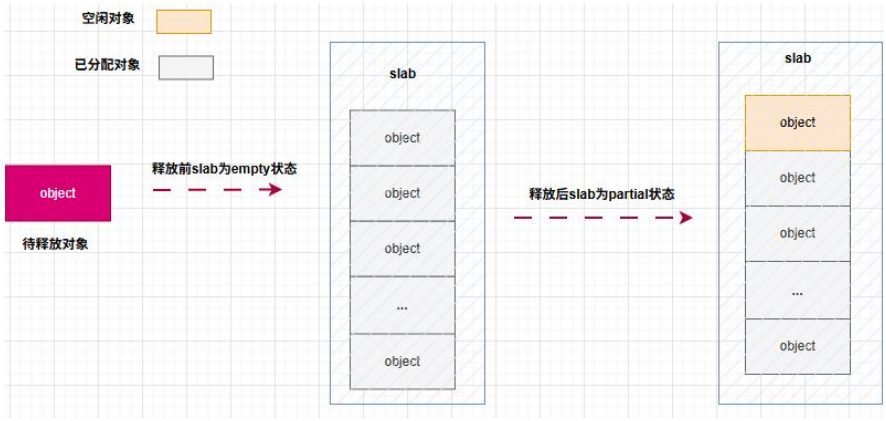


was_frozen 指向释放对象所属 slab 结构中的 frozen 属性，用来表示 slab 是否在 slab cache 的本地 cpu 缓存 partial 链表中。

```
was_frozen = new.frozen;
```

如果 was_frozen == 1 表示释放对象所属 slab 本来就在 kmem_cache_cpu->partial 链表中，内核将对象直接释放回 slab 中，slab 的原有位置不做改变。此时会跳过 if ((!new.inuse || !prior) && !was_frozen) 分支，执行 slab_update_freelist 函数，将对象释放回 slab 中。slab 的原有位置不做改变。这种情况下，不会发生 slab 节点的迁移。

10.3.3 释放对象后 slab 从 full slab 变为 partial slab



如果释放对象所属 slab 原来是一个 full slab，恰恰说明该 slab 拥有比较好的局部性，进程经常从该 slab 中分配对象，slab 十分活跃，才导致它变为了一个 full slab。此时 slab 是不在 slab cache 的本地 cpu 缓存 kmem_cache_cpu->partial 链表中。

随着对象的释放，该 slab 从一个 full slab 变为了 partial slab，内核为了更好的利用该 slab 的局部性，所以需要将该 slab 插入到 slab cache 的本地 cpu 缓存 kmem_cache_cpu->partial 链表中。

```
if (likely(!n)) {
        if (likely(was_frozen)) {
            stat(s, FREE_FROZEN);
        } else if (kmem_cache_has_cpu_partial(s) && !prior) {
          // 将 slub 插入到 kmem_cache_cpu 中的 partial 列表中
            put_cpu_partial(s, slab, 1);
            stat(s, CPU_PARTIAL_FREE);
        }
        return;
    }
```

会导致 n == null 的可能情况如下：

1. 如果 (!new.inuse || !prior) && !was_frozen 条件不满足，等价于下面任意一个条件不满足，都会导致 n == null :

    1) slab 处于冻结状态（was_frozen = true），无需操作节点链表。因为：冻结的 slab（was_frozen = true）不参与 NUMA 节点的 full/partial 链表迁移、空 slab 回收等操作，仅需完成对象的原子释放。

    2) slab 处于未冻结状态（was_frozen = false）,但是，new.inuse != 0 且 prior != NULL。其中，new.inuse != 0 表示释放后 slab 中 仍有已被使用的对象，此时处于 partial 状态。prior != NULL 表示释放前 slab 为 partial 状态，已有空闲对象。也就是说，这种情况下，slab 释放前后均为 partial 状态，且已挂载在节点 partial 链表上。无需迁移链表、无需回收 slab。

2. 如果 (!new.inuse || !prior) && !was_frozen 条件满足，但是 cpu 中不含有 partial 列表且 prior == null 时，会导致 n 为 null。

只有 prior == null 时，释放前 slab 的状态才为 full slab 。

如果此时，was_frozen == true，即 slab 冻结在当前 cpu 缓存 partial 链表中。不需要迁移 slab 节点。

如果此时 was_frozen == flase，并且 cpu 本地缓存 kmem_cache_cpu 结构中包含 partial 列表时，会将 slab 迁移到 slab cache 的本地 cpu 缓存 kmem_cache_cpu->partial 链表中。因为随着对象的释放，该 slab 从一个 full slab 变为了 partial slab，内核为了更好的利用该 slab 的局部性，需要将该 slab 迁移到到 slab cache 的本地 cpu 缓存 kmem_cache_cpu->partial 链表中。

```
struct kmem_cache_cpu {
    ......
    struct slab *partial;   /* Partially allocated frozen slabs */
    ......
}
```

将 slab 插入到 kmem_cache_cpu->partial 链表的逻辑封装在 put_cpu_partial 中，put_cpu_partial 函数最重要的一个考量逻辑是需要确保 kmem_cache_cpu->partial 链表中所有 slab 中包含的空闲对象总数不能超过 kmem_cache->cpu_partial 的限制。

```
/*
 * Put a slab into a partial slab slot if available.
 *
 * If we did not find a slot then simply move all the partials to the
 * per node partial list.
 */
static void put_cpu_partial(struct kmem_cache *s, struct slab *slab, int
drain)
{
    // 指向原有 kmem_cache_cpu 中的 partial 列表
    struct slab *oldslab;
    struct slab *slab_to_put = NULL;
    unsigned long flags;
    int slabs = 0;

    local_lock_irqsave(&s->cpu_slab->lock, flags);
     // 获取 slab cache 中原有的 cpu 本地缓存 partial 列表首页 slab
    oldslab = this_cpu_read(s->cpu_slab->partial);

    // 记录当前 CPU 局部缓存 partial 链表中的总 slab 数量
    //（链表中所有 slab 的计数总和）是否超过了 CPU 局部 partial 缓存的
    //最大容量（内核配置的阈值），如果超出 s->cpu_partial_slabs
    //则需要将 链表从 CPU 局部 partail 链表缓存（kmem_cache_cpu->partial)
    //中全部迁移到 NUMA 节点的 partial 链表上
    // 转移之后，再把当前 slub 插入到 kmem_cache_cpu->partial 列表中
    // 如果没有超过 s->cpu_partial ，则无需转移直接插入
    if (oldslab) {
        if (drain && oldslab->slabs >= s->cpu_partial_slabs) {
            //将旧的 partial slab 链表标记为「待迁移」，后续调用
            //__put_partials 函数做迁移
             slab_to_put = oldslab;
            //清空当前 CPU 局部缓存的旧链表头，为放入新 slab 腾出空间；
             oldslab = NULL;
        } else {
            // 读取旧缓存中的 slab 数量，后续用于累加
             slabs = oldslab->slabs;
        }
    }
    // 累加新 slab，更新缓存总数量
    slabs++;
    // 这个 slab 是待释放的 slab。要在 新 slab 中记录当前缓存的总数量
    slab->slabs = slabs;
    //将新 slab 作为链表头，指向旧缓存链表。
    slab->next = oldslab;
```

```
    // 原子写入。将新 slab 赋值为当前 CPU 局部 partial 缓存的新链表头，
    //完成缓存更新
    this_cpu_write(s->cpu_slab->partial, slab);
    //释放当前 CPU 的本地锁，并恢复之前保存的中断状态（开启本地中断）
    local_unlock_irqrestore(&s->cpu_slab->lock, flags);
    if (slab_to_put) {
    //实际执行迁移操作。
    //将标记的旧 slab 链表从 CPU 局部缓存迁移到 NUMA 节点的 partial 链表；
        __put_partials(s, slab_to_put);
        stat(s, CPU_PARTIAL_DRAIN);
    }
}
```

当前 CPU 局部缓存 partial 链表中总的 slab 数量（链表中所有 slab 的计数总和）声明如下：

```
struct slab {
           ......
           //slab 所在管理链表中的包含的 slab 总数
           int slabs;
          ......
}
```

CPU 局部 partial 缓存的最大容量（内核配置的阈值）声明如下：

```
struct kmem_cache {
           ......
         //slab cache 中每个 cpu 本地缓存 partial 链表中包含 slab 的总数
            unsigned int cpu_partial_slabs;
           ......
}
```

slab 在内核中的数据结构用 struct slab 中的相关结构体表示，slab 在 slab cache 架构中一般是由 kmem_cache_cpu->partial 链表和 kmem_cache_node->partial 链表来组织管理。

partial 链表中包含的 slab 个数，存储在 parital 链表中首个 slab 结构中的 slabs 属性和 pobjects 属性中。

在 put_cpu_partial 函数的开始，内核直接获取 parital 链表的首个 slab —— oldpage，并通过 oldslab->slabs 与 s->cpu_partial_slabs 比较，来判断当前 kmem_cache_cpu->partial 链表中包含的空闲 slab 总数是否超过了 kmem_cache 结构中规定的阈值。

如果超过了，则通过__put_partials 转移 kmem_cache_cpu->partial 链表中的所有 slab 到对应的 kmem_cache_node->partial 链表中。

既然 kmem_cache_cpu->partial 链表有容量的限制，那么同样 kmem_cache_node->partial 链表中的容量也会有限制。kmem_cache_node->partial 链表中所包含 slab 个数的上限由 kmem_cache 结构中的 min_partial 属性决定。

```
  struct kmem_cache {
      ......
```

```
    // slab cache 在 numa node 中缓存的 slab 个数上限，
    //slab 个数超过该值，空闲的 empty slab 则会被回收至伙伴系统
    unsigned long min_partial;
    ......

}
```

__put_partials 是将 CPU 局部 partial 缓存的 slab 链表迁移到 NUMA 节点全局 partial 链表的核心函数。

```
static void __put_partials(struct kmem_cache *s, struct slab *partial_slab)
{
    //n:当前持有锁的 NUMA 节点
    //n2：当前遍历的 slab 所属的 NUMA 节点
    struct kmem_cache_node *n = NULL, *n2 = NULL;
    //待销毁的空 slab 链表头。用于将需要释放回伙伴系统的 slub 串联起来
    //后续统一将 discard_page 链表中的 slub 释放回伙伴系统
    struct slab *slab, *slab_to_discard = NULL;
    unsigned long flags = 0;
   //采用「遍历单向链表」的方式，逐个处理从 CPU 局部缓存迁移出来的 slab
    while (partial_slab) {
        slab = partial_slab;
        // 保存下一个 slab
        partial_slab = slab->next;
        n2 = get_node(s, slab_nid(slab));
        if (n != n2) {
           //若当前 slab 所属节点与上一个持锁节点不一致
            if (n)
                 //存在上一个持有锁的节点，释放这个持有锁的节点
                spin_unlock_irqrestore(&n->list_lock, flags);
            // 当前节点设为持锁节点
            n = n2;
            //当前节点加锁
            spin_lock_irqsave(&n->list_lock, flags);
        }
        if (unlikely(!slab->inuse && n->nr_partial >= s->min_partial)) {
            //若 slab 为空，且节点 partial 链表已满。进入此分支
            //当前空 slab 的 next 指针指向待销毁链表的旧头
            slab->next = slab_to_discard;
            //更新 slab_to_discard 为当前 slab
            slab_to_discard = slab;
        } else {
           //将 slab 挂载到 NUMA 节点 n 的全局 partial 链表尾部
            add_partial(n, slab, DEACTIVATE_TO_TAIL);
            stat(s, FREE_ADD_PARTIAL);
```

```
        }
    }
    //遍历完所有待迁移 slab 后，若之前持有节点锁（n != NULL），
    //则释放锁并恢复本地中断状态；
    if (n)
        spin_unlock_irqrestore(&n->list_lock, flags);

    //遍历待销毁链表，批量回收空 slab
    while (slab_to_discard) {
        slab = slab_to_discard;
        // 先保存下一个节点，避免链表断裂
        slab_to_discard = slab_to_discard->next;
        stat(s, DEACTIVATE_EMPTY);
        //核心操作：释放空 slab 的物理页到伙伴系统
        discard_slab(s, slab);
        stat(s, FREE_SLAB);
    }
}
```

遍历待迁移 slab 链表，逐个处理从 CPU 局部缓存迁移出来的 slab。如果当前要转移的 slab 是一个 empty slab，slab->inuse == 0 ，并且此时 kmem_cache_node->partial 链表所包含的 slab 个数 kmem_cache_node->nr_partial 已经超过了 kmem_cache-> min_partial 的限制，那么内核就会直接将这个 empty slab 释放回伙伴系统中。

10.2.4 释放对象后 slab 从 partial slab 变为 empty slab

如果释放对象所在的 slab 原来是一个 partial slab ，由于对象的释放刚好变成了一个 empty slab。

若释放前 prior != NULL 时，slab 处于 partial 状态。如果 was_frozen == 0。走

```
    do {
        if (unlikely(n)) {
            spin_unlock_irqrestore(&n->list_lock, flags);
            n = NULL;
        }
           ......
        if ((!new.inuse || !prior) && !was_frozen) {
            if (!kmem_cache_has_cpu_partial(s) || prior) {
              //prior != NULL， 且 was_frozen == 0.时会走这个分支
                n = get_node(s, slab_nid(slab));
                spin_lock_irqsave(&n->list_lock, flags);
                on_node_partial = slab_test_node_partial(slab);
            }
        }
    } while (!slab_update_freelist(s, slab,
```

```
        prior, counters,
        head, new.counters,
        "__slab_free"));
```

若 on_node_partial ==false，slab 不在 NUMA 节点全局 partial 链表上接着走：

```
    if (prior && !on_node_partial) {
        spin_unlock_irqrestore(&n->list_lock, flags);
        return;
```

直接返回，不需要迁移 slab 节点。

若 on_node_partial == true，slab 在 NUMA 节点全局 partial 链表上，接着走：

```

    if (unlikely(!new.inuse && n->nr_partial >= s->min_partial))
        goto slab_empty;

```

因为释放后为 empty 状态，所以 new.inuse == 0，此时还需要检查当前 partial 链表中的容量 kmem_cache_node->nr_partial 不能超过 kmem_cache-> min_partial 的限制。如果超过限制了，直接将这个 empty slab 释放回伙伴系统中。

```
slab_empty:
    if (prior) {
        remove_partial(n, slab);
        stat(s, FREE_REMOVE_PARTIAL);
    } else {
        remove_full(s, n, slab);
    }

    spin_unlock_irqrestore(&n->list_lock, flags);
    stat(s, FREE_SLAB);
    discard_slab(s, slab);
```

如果，没有超过限制，因为释放对象所属的 slab 本来就在 kmem_cache_node->partial 链表中，这种情况下就是直接释放对象回 slab 中，也无需改变 slab 的位置。

10.3 销毁 slab cache

slab cache 的销毁过程和 slab cache 的创建过程相反。slab cache 销毁的核心步骤如下：

1.首先需要释放 slab cache 在所有 cpu 中的缓存 kmem_cache_cpu 中占用的资源，包括被 cpu 缓存的 slab (kmem_cache_cpu->slab)，以及 kmem_cache_cpu->partial 链表中缓存的所有 slab，将它们统统归还到伙伴系统中。

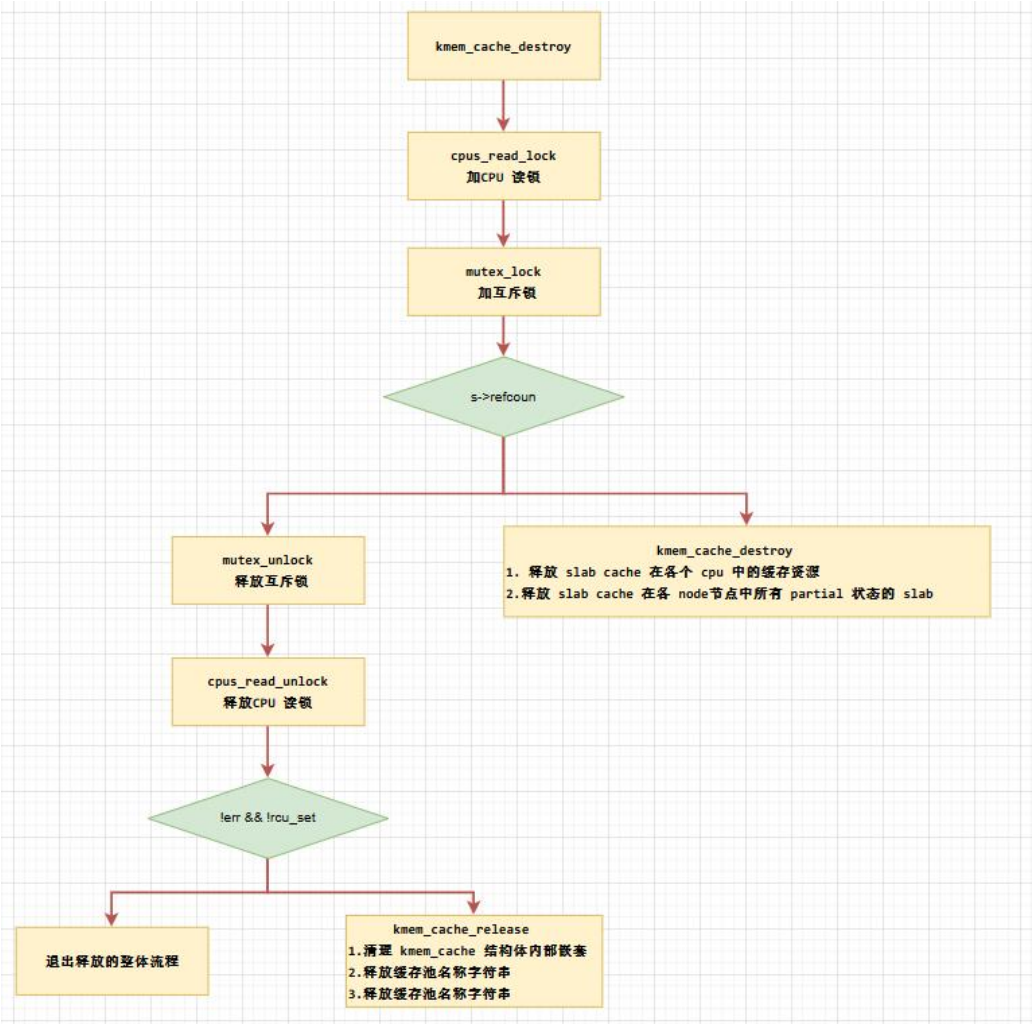
2.释放 slab cache 在所有 NUMA 节点中的缓存 kmem_cache_node 占用的资源，也就是将 kmem_cache_node->partial 链表中缓存的所有 slab ，统统释放回伙伴系统中。

3.在 sys 文件系统中移除 /sys/kernel/slab/<cacchename> 节点相关信息。

4.从 slab cache 的全局列表中删除该 slab cache。

5.释放 kmem_cache_cpu 结构，kmem_cache_node 结构，kmem_cache 结构。

kmem_cache_destroy 函数整体执行流程如下:




```
void kmem_cache_destroy(struct kmem_cache *s)
{
    int err = -EBUSY;
    bool rcu_set;

    if (unlikely(!s) || !kasan_check_byte(s))
        return;
    cpus_read_lock();
     // 获取 slab cache 链表的全局互斥锁
    mutex_lock(&slab_mutex);
    rcu_set = s->flags & SLAB_TYPESAFE_BY_RCU;
    // 将 slab cache 的引用计数减 1
    s->refcount--;
    if (s->refcount)
     // 如果该 slab cache 还存在引用，则不能销毁，跳转到 out_unlock 分支
        goto out_unlock;
   // 销毁 memory cgroup 相关的 cache
```

```
    err = shutdown_cache(s);
    WARN(err, "%s %s: Slab cache still has objects when called from %pS",
         __func__, s->name, (void *)_RET_IP_);
out_unlock:
   // 释放 slab_mutex 互斥锁
    mutex_unlock(&slab_mutex);
    //释放 cpus_read_lock CPU 读锁
    cpus_read_unlock();
    if (!err && !rcu_set)
    //
        kmem_cache_release(s);
}
EXPORT_SYMBOL(kmem_cache_destroy);
```

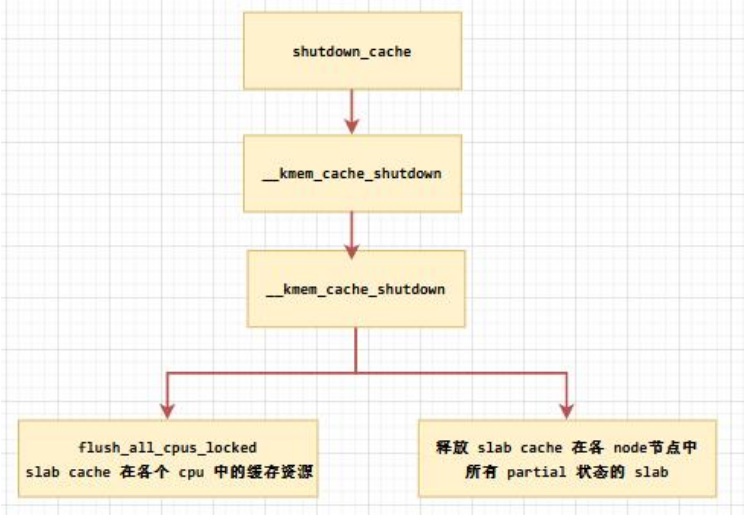
在开始正式销毁 slab cache 之前，首先需要将 slab cache 的引用计数 refcount 减 1。并需要判断 slab cache 是否还存在其他地方的引用。创建流程中得知，当利用 kmem_cache_create 创建 slab cache 的时候，内核会检查当前系统中是否存在一个各项参数和我们要创建 slab cache 参数差不多的一个 slab cache，如果存在，那么内核就不会再继续创建新的 slab cache，而是复用已有的 slab cache。

一个可以被复用的 slab cache 需要满足以下四个条件：

1. 指定的 slab_flags_t 相同。
2. 指定对象的 object size 要小于等于已有 slab cache 中的对象 size（kmem_cache->size）。
3. 如果指定对象的 object size 与已有 kmem_cache->size 不相同，那么它们之间的差值需要再一个 word size 之内。
4. 已有 slab cache 中的 slab 对象对齐 align （kmem_cache->align）要大于等于指定的 align 并且可以整除 align 。 。

随后会在 sys 文件系统中为复用 slab cache 起一个别名 alias 并创建一个 /sys/kernel/slab/aliasname 目录，但是该目录下的文件需要软链接到原有 slab cache 在 sys 文件系统对应目录下的文件。这里的 aliasname 就是通过 kmem_cache_create 指定的 slab cache 名称。

在这种情况，系统中的 slab cache 就可能在多个地方产生引用，所以在销毁的时候需要判断这一点。如果存在其他地方的引用，则需要停止销毁流程，如果没有其他地方的引用，则调用开始正式的销毁流程。首先调用 shutdown_cache 函数释放 slab cache 在各个 cpu 中的缓存资源 和 释放 slab cache 在各 node 节点中所有 partial 状态的 slab。




```
    static int shutdown_cache(struct kmem_cache *s)
{
    /* free asan quarantined objects */
    kasan_cache_shutdown(s);
     //释放 slab cache 占用的所有资源
    if (__kmem_cache_shutdown(s) != 0)
        return -EBUSY;
     // 从 slab cache 的全局列表中删除该 slab cache
    list_del(&s->list);
    //启用 SLAB_TYPESAFE_BY_RCU 标志位（RCU 延迟销毁）
    //SLAB_TYPESAFE_BY_RCU 表示该缓存池支持「RCU（Read-Copy-Update）
    //类型安全」，即允许在 RCU 读临界区内持有缓存池对象，无需立即释放；
    if (s->flags & SLAB_TYPESAFE_BY_RCU) {
    //将待销毁的 kmem_cache 添加到 slab_caches_to_rcu_destroy
    //专用链表尾部，该链表用于存放需要 RCU 延迟销毁的缓存池；
        list_add_tail(&s->list, &slab_caches_to_rcu_destroy);
    //调度一个内核工作队列（workqueue）任务，
    //该任务会在 RCU 宽限期（grace period）结束后，
    //统一销毁 slab_caches_to_rcu_destroy 链表中的所有缓存池，
    //确保所有 RCU 读临界区已退出，无悬空引用；
        schedule_work(&slab_caches_to_rcu_destroy_work);
    } else {
    //未启用 SLAB_TYPESAFE_BY_RCU 标志位，无需延迟销毁，
    //直接执行剩余的收尾清理操作，释放相关资源；
        kfence_shutdown_cache(s);
        debugfs_slab_release(s);
    }
    return 0;
}
```

负责执行缓存池释放的核心逻辑，封装在__kmem_cache_shutdown 函数中：

```
/*
 * Release all resources used by a slab cache.
 */
int __kmem_cache_shutdown(struct kmem_cache *s)
{
    int node;
    struct kmem_cache_node *n;
    //清理所有 CPU 的局部 kmem_cache_cpu 缓存，
    //并通过锁定机制保证清理的完整性，避免 CPU 局部缓存残留资源
    //导致后续校验失败
    flush_all_cpus_locked(s);
     // 释放 slab cache 中 numa 节点缓存 kmem_cache_node
    //  中 partial 列表上的所有 slub
    for_each_kmem_cache_node(s, node, n) {
    //释放当前 NUMA 节点 n 上的所有 partial 状态 slab，
    //回收其占用的物理页资源；
        free_partial(s, n);
        //判断当前节点是否有残留资源
        //n->nr_partial：当前 NUMA 节点 n 上剩余的 partial slab 数量
        //node_nr_slabs(n)：当前 NUMA 节点 n 上的总 slab 数量
        //（包括 full 状态、partial 状态等所有类型的 slab，
        //是节点内 slab 的总数统计）
        if (n->nr_partial || node_nr_slabs(n))
            return 1;
    }
    return 0;
}
```

10.3.1 释放 slab cache 在各个 cpu 中的缓存资源

释放 slab cache 在各个 cpu 中的缓存资源的主要逻辑封装在 flush_all_cpus_locked 函数中。其核心执行流程是：锁调试断言 → 获取全局 flush 锁 → 第一遍遍历（提交清理工作项）→ 第二遍遍历（等待清理完成）→ 释放全局 flush 锁。

第一遍遍历（异步提交）：遍历过程中通过 has_cpu_slab 判断 slab cache 是否在该 cpu 中还占有缓存资源，如果是则调用 queue_work_on 快速向所有需要清理的 CPU 提交工作项，实现多 CPU 并行清理，提升效率；

第二遍遍历（同步等待）：调用 flush_work 函数阻塞当前进程，确保所有 CPU 的清理工作都执行完毕，为后续 __kmem_cache_shutdown 的 NUMA 节点清理提供「无局部缓存残留」的干净环境；

具体函数 flush_all_cpus_locked 源码如下：

```
static void flush_all_cpus_locked(struct kmem_cache *s)
```

```
{
    struct slub_flush_work *sfw;
    unsigned int cpu;

    lockdep_assert_cpus_held();
    //获取全局 flush 锁
    mutex_lock(&flush_lock);
    //遍历所有在线 CPU，进行资源的释放（异步提交清理工作）
    for_each_online_cpu(cpu) {
    //通过 per_cpu 宏获取编号为 cpu 的 CPU 对应的 slub_flush 结构体指针，
    //每个 CPU 独立拥有该结构体，无跨 CPU 竞争。
        sfw = &per_cpu(slub_flush, cpu);
        //检查 cpu 上是否还有 slab cache 的相关缓存资源
        if (!has_cpu_slab(cpu, s)) {
           // cpu 上是无 slab cache 的相关缓存资源。走此逻辑。
          //设置 sfw->skip = true（标记后续无需等待该 CPU 的清理工作），
          //直接跳过当前 CPU，避免无效操作；
            sfw->skip = true;
            continue;
        }
        //初始化 sfw 中的属性 work，将其与实际执行清理的函数
        // flush_cpu_slab 绑定（flush_cpu_slab 是 CPU 局部 slab
        //清理的核心实现函数），工作项初始化后才能被提交到工作队列。
        INIT_WORK(&sfw->work, flush_cpu_slab);
      //清除跳过标记，标记该 CPU 需要执行清理工作，后续需等待其完成；
        sfw->skip = false;
     //将待清理的 kmem_cache 缓存池指针赋值给 sfw，
    //这个指针就是 flush_cpu_slab 函数清理的目标
        sfw->s = s;
    //提交 work 任务到指定 CPU 工作队列
    //flushwq：将工作项提交到 SLUB 专用工作队列，
    //避免与其他内核工作（如磁盘 IO、网络协议栈）竞争；
    //作用：异步发起清理工作，当前进程不阻塞，直接继续遍历下一个 CPU，
    //实现多 CPU 并行清理。
        queue_work_on(cpu, flushwq, &sfw->work);
    }
    //遍历所有在线 CPU，进行资源的释放（同步等待清理完成）
    //遍历所有在线 CPU，等待每个需要清理的 CPU 工作项执行完毕
    for_each_online_cpu(cpu) {
    //重新获取当前 CPU 的 slub_flush_work 结构体
        sfw = &per_cpu(slub_flush, cpu);
        if (sfw->skip)
         // CPU 被标记为 skip=true（无对应局部缓存），
        //直接跳过，无需等待
```

```
            continue;
        //阻塞当前进程，直到 sfw->work 对应的 flush_cpu_slab 函数执行完毕，
        //用来确保该 CPU 的局部 slab 缓存已彻底清理。
        flush_work(&sfw->work);
    }
    //释放 锁
    mutex_unlock(&flush_lock);
}
```

```
static bool has_cpu_slab(int cpu, struct kmem_cache *s)
{
     // 获取 cpu 在 slab cache 上的本地缓存
    struct kmem_cache_cpu *c = per_cpu_ptr(s->cpu_slab, cpu);
      // 判断 cpu 本地缓存中是否还有缓存的 slub
    return c->slab || slub_percpu_partial(c);
}
```

10.3.2 释放 slab cache 在各 node 节点中所有 partial 状态的 slab

释放 slab cache 在各 node 节点中所有 partial 状态的 slab 的核心执行流程封装在 __kmem_cache_shutdown 函数中的 for_each_kmem_cache_node 循环中。

循环的主要执行流程为：遍历所有 NUMA 节点，筛选出与待释放 slab cache 关联的所有 NUMA 节点。调用 free_partial 逐个释放该节点上的 artial 列表上的所有 slab。如果所有节点上的 partial 表的 slab 都已释放完毕，没有残余资源。继续执行后续释放流程。如果，任意一个节点调用 free_partial 函数后，仍存在残留资源，返回 1，终止后续释放流程。

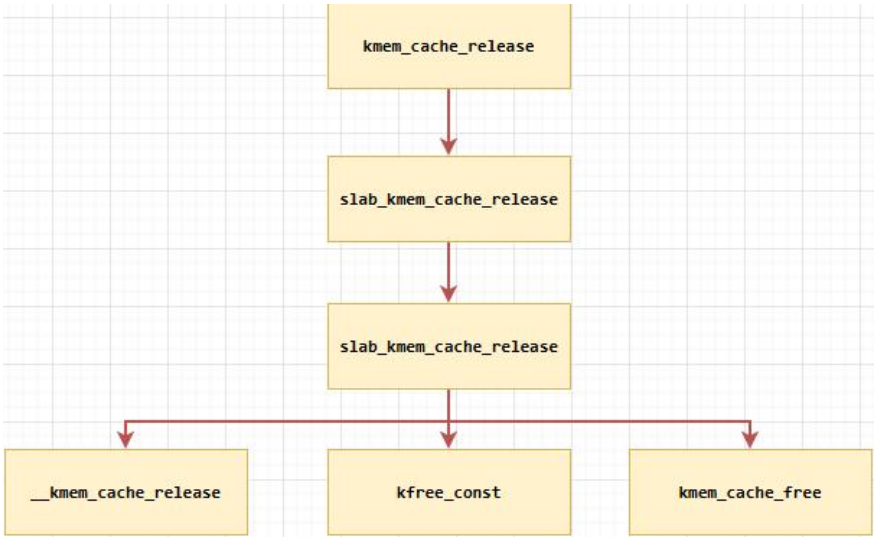
```
    // 释放与待释放 slab cache 相关的  numa 节点缓存中的 kmem_cache_node
    //  partial 列表上的所有 slub
    for_each_kmem_cache_node(s, node, n) {
    //释放当前 NUMA 节点 n 上的所有 partial 状态 slab，
    //回收其占用的物理页资源；
        free_partial(s, n);
        //判断当前节点是否有残留资源
        //n->nr_partial：当前 NUMA 节点 n 上剩余的 partial slab 数量
        //node_nr_slabs(n)：当前 NUMA 节点 n 上的总 slab 数量
        //（包括 full 状态、partial 状态等所有类型的 slab，
        //是节点内 slab 的总数统计）
        if (n->nr_partial || node_nr_slabs(n))
          //此分支，表示缓存池清理后仍存在残留资源。
            return 1;
    }
```

free_partial 函数源码实现如下：

```
/*
 * Attempt to free all partial slabs on a node.
 * This is called from __kmem_cache_shutdown(). We must take list_lock
 * because sysfs file might still access partial list after the shutdowning.
 */
static void free_partial(struct kmem_cache *s, struct kmem_cache_node *n)
{
    LIST_HEAD(discard);
    struct slab *slab, *h;
    //获取节点自旋锁并禁用中断
    BUG_ON(irqs_disabled());
    spin_lock_irq(&n->list_lock);
    //遍历 n->partial 链表
    //对每个 slab 分两种场景处理

    list_for_each_entry_safe(slab, h, &n->partial, slab_list) {
        if (!slab->inuse) {
        //场景 1: slab 为空。
       //将 slab 从 n->partial 链表中移除，递减 n->nr_partial 计数器
            remove_partial(n, slab);
      //将空 slab 添加到局部 discard 临时链表，暂存等待后续批量销毁，
      //不占用临界区时间。
            list_add(&slab->slab_list, &discard);
        } else {
        //场景 2: slab 非空
        //打印残留对象的内存地址、数量等信息
            list_slab_objects(s, slab,
              "Objects remaining in %s on __kmem_cache_shutdown()");
        }
    }
    //释放节点自旋锁并启用中断
    spin_unlock_irq(&n->list_lock);
    //遍历局部 discard 临时链表，对每个空 slab
    //调用 discard_slab(s, slab) 执行最终销毁:
    list_for_each_entry_safe(slab, h, &discard, slab_list)
    //将 slab 对应的物理页归还给内核伙伴系统（通过 __free_pages），
    //清理 slab 元数据（如 slab 结构体、空闲链表等），
    //彻底终止 slab 生命周期;
        discard_slab(s, slab);
}
```

10.3.3 释放 slab cache 的核心数据结构



slab cache 核心数据结构的释放流程封装在 kmem_cache_release 函数中。总体分为三部分：

1. 清理 kmem_cache 结构体内部嵌套
2. 释放缓存池名称字符串
3. 释放缓存池名称字符串

只有当缓存池已无残留资源并且缓存池未启用 SLAB_TYPESAFE_BY_RCU 标志位时才会进入 slab cache 的核心数据结构的释放流程。这是 释放 slab cache 的最后一个核心流程。

```
    if (!err && !rcu_set)
        kmem_cache_release(s);
```

!err：err == 0，表示 shutdown_cache 关闭成功，缓存池已无残留资源。

!rcu_set：rcu_set == false，表示缓存池未启用 SLAB_TYPESAFE_BY_RCU 标志位

```
static void kmem_cache_release(struct kmem_cache *s)
{
    slab_kmem_cache_release(s);
}
```

```
void slab_kmem_cache_release(struct kmem_cache *s)
{
   //清理 kmem_cache 结构体内部嵌套的关联元数据，
```

```
// slab cache 中的 kmem_cache_cpu 结构以及 kmem_cache_node 结构
    __kmem_cache_release(s);
    //释放缓存池名称字符串
    kfree_const(s->name);
    //最后释放 slab cache 的核心数据结构 kmem_cache
    //也就是 kmem_cache 结构体自身内存，
    //标志着 kmem_cache 结构体自身内存的彻底释放
    kmem_cache_free(kmem_cache, s);
}
```